

✓ JavaScript Mastery List (Pre-Framework)

1. Language Fundamentals

- Variables: `var`, `let`, `const`
 - Mutation vs reassignment
 - Primitive types
 - Reference types
 - `typeof` quirks
 - Truthy & falsy values
 - Implicit type coercion
 - Explicit type conversion
 - `==` vs `===`
 - `Object.is`
 - Strict mode (`"use strict"`)
-

2. Scope & Execution Model

- Single-threaded nature of JS
 - Call stack
 - Heap memory
 - Execution context
 - Global execution context
 - Function execution context
 - Lexical environment
 - Scope chain
 - Lexical scope vs dynamic scope
 - Block scope vs function scope
 - Hoisting rules
 - Temporal Dead Zone (TDZ)
 - Garbage collection basics
-

3. Functions (Deep)

- Function declarations vs expressions
- Arrow functions (limitations)
- Default parameters
- Rest parameters
- Spread operator

- Higher-order functions
 - Pure functions vs side effects
 - Closures
 - Closure patterns (module, private state)
 - Currying
 - Partial application
 - Function composition
 - Recursion
 - Stack overflow scenarios
 - IIFE
 - Generator functions
-

4. `this` & Context

- `this` binding rules
 - Global binding
 - Implicit binding
 - Explicit binding
 - `new` binding
 - `call`, `apply`, `bind`
 - `this` in arrow functions
 - Losing `this` in callbacks
-

5. Objects & Prototypes

- Object creation patterns:
 - Factory functions
 - Constructor functions
 - `Object.create`
 - Prototype chain
 - `__proto__` vs `prototype`
 - Constructor functions
 - `new` keyword internals
 - ES6 classes (syntax sugar)
 - Inheritance patterns (prototypal)
 - `instanceof`
 - Property descriptors
 - Getters & setters
 - Enumerability & property lookup
-

6. Arrays & Data Structures

- Array internals
 - Sparse arrays
 - Mutation vs non-mutation methods
 - `map`
 - `filter`
 - `reduce`
 - `find`
 - `findIndex`
 - `some`
 - `every`
 - `flat`
 - `flatMap`
 - `sort` (comparison pitfalls)
 - `slice` vs `splice`
 - Array destructuring
 - `Set`
 - `Map`
 - `WeakSet`
 - `WeakMap`
-

7. Strings, Numbers & Dates

- String immutability
 - Template literals
 - UTF-16 basics
 - Regular expressions (fundamentals)
 - Floating-point precision issues
 - `NaN`, `Infinity`
 - `Number.EPSILON`
 - `parseInt` pitfalls
 - `Math` object
 - Date & time handling
 - UTC & time zones
-

8. Asynchronous JavaScript (Critical)

- Blocking vs non-blocking
- Callbacks
- Callback hell

- Event loop architecture
 - Web APIs concept
 - Task queue (macrotasks)
 - Microtask queue
 - Promise state machine
 - Promise chaining
 - Error propagation in promises
 - `then`, `catch`, `finally`
 - `async` / `await`
 - Error handling with `try/catch`
 - `Promise.all`
 - `Promise.race`
 - `Promise.allSettled`
 - `Promise.any`
 - Race conditions
-

9. Modules & Code Organization

- ES Modules
 - `import` / `export`
 - Named vs default exports
 - Module scope
 - Live bindings
 - Circular dependencies
 - CommonJS vs ESM
 - Tree-shaking concept
-

10. Error Handling & Debugging

- `try` / `catch` / `finally`
 - `throw`
 - Built-in error types
 - Custom error classes
 - Stack traces
 - Sync vs async errors
 - Debugging with breakpoints
 - Console methods (proper use)
-

11. Data Handling & Immutability

- Shallow vs deep copy
 - Reference pitfalls
 - Immutability principles
 - `Object.keys`
 - `Object.values`
 - `Object.entries`
 - `Object.assign`
 - `Object.freeze`
 - `Object.seal`
 - JSON serialization
 - `JSON.parse`
 - `JSON.stringify`
-

12. DOM & Browser Concepts (Even for React)

- DOM tree
 - Query selectors
 - Event listeners
 - Event delegation
 - Event bubbling
 - Event capturing
 - Default browser actions
 - `preventDefault`
 - `stopPropagation`
 - Creating/modifying/removing elements
 - Browser rendering pipeline
 - Reflow vs repaint
-

13. Runtime APIs

- `setTimeout`
 - `setInterval`
 - `requestAnimationFrame`
 - `fetch`
 - `AbortController`
 - URL & query params
 - `localStorage`
 - `sessionStorage`
 - Cookies (basics)
-

14. Performance & Memory

- Memory leaks
 - Closure retention
 - Event listener leaks
 - Time complexity awareness
 - Space complexity awareness
 - Debouncing
 - Throttling
 - Performance profiling basics
-

15. Modern JavaScript Patterns

- Functional programming principles
 - Declarative vs imperative code
 - Composition over inheritance
 - Observer pattern
 - Callback patterns
 - Separation of concerns
 - Factory pattern
 - Revealing module pattern
-

16. Tooling Awareness (Pre-Framework)

- npm basics
- `package.json`
- Semantic versioning
- Dependencies vs devDependencies
- Bundlers (conceptual)
- Transpilation (Babel concept)
- Linting (ESLint concept)